Tutorial Prático: Chat Privado em Tempo Real

Spring Boot, WebSocket e STOMP (Comunicação Ponto a Ponto) • Pacote br.edu.ifsp.bra.websocket_queue_chat

Objetivo da Aula

Objetivo Didático: Construir e compreender uma aplicação funcional de mensagens privadas (User1 User2) em tempo real. Você aprenderá a interceptar o handshake HTTP Upgrade, configurar o Message Broker para filas privadas (/queue e /user) e manipular o envio programático via SimpMessagingTemplate.

1. Orientação Inicial: Criando o Projeto Spring Boot

Antes de iniciar a codificação, o estudante deve criar um novo projeto Spring Boot. Isso pode ser feito via **Spring Initializr** (start.spring.io) ou diretamente na sua IDE de preferência (IntelliJ IDEA, Eclipse ou VS Code):

- **Group:** br.edu.ifsp.bra | **Artifact:** websocket_queue_chat
- **Linguagem:** Java (versão 17 ou superior) | **Gerenciador:** Maven
- **Dependências Obrigatórias:** Adicione as dependências **Spring Web** e **WebSocket** (spring-boot-starter-websocket) e opcionalmente **Lombok**.

2. Estrutura do Projeto Spring Boot (Árvore de Arquivos)

Organize as classes do seu projeto respeitando a estrutura de pacotes abaixo:

```
meu-projeto-chat/
├── pom.xml
└── src/
    └── main/
        ├── java/
        │   └── br/edu/ifsp/bra/websocket_queue_chat/
        │       ├── CustomHandshakeHandler.java  <-- (1) Identifica usuário no Handshake
        │       ├── MessageDTO.java              <-- (2) DTO do JSON da mensagem
        │       ├── WebSocketConfig.java         <-- (3) Configura o Broker e Endpoints
        │       └── PrivateChatController.java   <-- (4) Roteia mensagens privadas
        └── resources/
            ├── application.properties
            └── static/
                └── index.html                   <-- (5) Cliente Web (SockJS / STOMP)
```

3. Dependências do Projeto (pom.xml)

Certifique-se de que seu arquivo pom.xml inclua o starter do WebSocket:

pom.xml

```xml
<dependencies>
    <!-- Suporte a WebSocket e Mensageria STOMP no Spring Boot -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-websocket</artifactId>
    </dependency>

    <!-- Lombok (Opcional, para Getters/Setters) -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

4. Identificação do Usuário no Handshake (CustomHandshakeHandler.java)

Conceito: O WebSocket inicia com uma requisição HTTP Upgrade. Para que o Spring saiba entregar mensagens privadas para um usuário específico (ex: User1), o CustomHandshakeHandler lê o parâmetro ?username= da URL e associa um Principal à sessão.

CustomHandshakeHandler.java

```
package br.edu.ifsp.bra.websocket_queue_chat;

import java.security.Principal;
import java.util.Map;
import org.springframework.http.server.ServerHttpRequest;
import org.springframework.web.socket.WebSocketHandler;
import org.springframework.web.socket.server.support.DefaultHandshakeHandler;
import org.springframework.web.util.UriComponentsBuilder;

public class CustomHandshakeHandler extends DefaultHandshakeHandler {

    @Override
    protected Principal determineUser(ServerHttpRequest request,
                                      WebSocketHandler wsHandler,
                                      Map<String, Object> attributes) {
        // Extrai o parâmetro ?username= da URL de conexão
        String username = UriComponentsBuilder.fromUri(request.getURI())
                .build()
                .getQueryParams()
                .getFirst("username");

        // Associa o Principal ao nome recebido ou gera ID genérico
        return () -> (username != null && !username.isBlank()
                      ? username
                      : "User_" + System.currentTimeMillis());
    }
}
```

5. Modelo de Dados da Mensagem (MessageDTO.java)

Conceito: As mensagens trafegam em formato JSON e são desserializadas automaticamente pelo Spring no objeto Java MessageDTO.

MessageDTO.java

```
package br.edu.ifsp.bra.websocket_queue_chat;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@NoArgsConstructor
@Data
@AllArgsConstructor
public class MessageDTO {
    private String sender;    // Quem enviou (ex: "User1")
    private String recipient; // Quem vai receber (ex: "User2")
    private String content;   // Conteúdo do texto
}
```

6. Configuração do Broker (WebSocketConfig.java)

Conceito: Habilita o broker em memória para /queue, estabelece /app para entrada nos Controllers, define /user para mensagens privadas e registra o CustomHandshakeHandler.

WebSocketConfig.java

```java
package br.edu.ifsp.bra.websocket_queue_chat;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        // Habilita a fila simples para mensagens privadas
        registry.enableSimpleBroker("/queue");

        // Prefixo para métodos anotados com @MessageMapping
        registry.setApplicationDestinationPrefixes("/app");

        // Prefixo para mensagens direcionadas a um usuário (/user)
        registry.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // Registra o endpoint /ws com CORS flexível e CustomHandshakeHandler
        registry.addEndpoint("/ws")
                .setAllowedOriginPatterns("*")
                .setHandshakeHandler(new CustomHandshakeHandler())
                .withSockJS();
    }
}
```

7. Controlador do Chat Privado (PrivateChatController.java)

Conceito: Injeta o SimpMessagingTemplate e utiliza convertAndSendToUser() para rotear o pacote exclusivamente para o destinatário e para o remetente.

PrivateChatController.java

```java
package br.edu.ifsp.bra.websocket_queue_chat;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Controller;

@Controller
public class PrivateChatController {

    @Autowired
    private SimpMessagingTemplate template;

    // Cliente envia para: /app/chat
    @MessageMapping("chat")
    public void processChatMessage(@Payload MessageDTO message) {
        // 1. Envia para a fila privada do DESTINATÁRIO
        template.convertAndSendToUser(
            message.getRecipient(),
            "/queue/private",
            message
        );

        // 2. Envia para a fila privada do REMETENTE (confirmação na tela)
        template.convertAndSendToUser(
            message.getSender(),
            "/queue/private",
            message
        );
    }
}
```

8. Cliente Web (src/main/resources/static/index.html)

Conceito: Conecta via SockJS passando o nome do usuário na URL (/ws?username=...) e se inscreve na fila privada /user/queue/private.

index.html (Parte 1/2: Estrutura HTML)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Chat Direct (User1 -> User2)</title>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/sockjs-client/1.6.1/sockjs.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/stomp.js/2.3.3/stomp.min.js"></script>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; line-height: 1.5; }
        .box { background: #f4f4f4; padding: 12px; border-radius: 5px; margin-bottom: 10px; width: 380px; }
        input[type="text"] { width: 92%; padding: 6px; margin: 4px 0; }
        button { padding: 6px 12px; background: #28a745; color: white; border: none; cursor: pointer; }
        #chatLog { background: #fff; border: 1px solid #ccc; height: 180px; overflow-y: scroll; padding: 8px;
}
    </style>
</head>
<body>
    <h2>Chat Direct (User1 -> User2)</h2>

    <div class="box">
        <h3>1. Conectar</h3>
        <input type="text" id="myUsername" placeholder="Seu Nome (ex: User1)">
        <input type="text" id="serverIp" placeholder="IP do Servidor" value="localhost">
        <button onclick="conectar()">Conectar ao Servidor</button>
    </div>

    <div class="box">
        <h3>2. Enviar Mensagem</h3>
        <input type="text" id="recipient" placeholder="Destinatário (ex: User2)">
        <input type="text" id="message" placeholder="Digite a mensagem...">
        <button onclick="enviar()">Enviar Mensagem</button>
    </div>

    <div class="box" style="width: 480px;">
        <h3>Conversa</h3>
        <div id="chatLog"></div>
    </div>
```

index.html (Parte 2/2: JavaScript & STOMP)

```
<script>
        let stompClient = null;

        function conectar() {
            const username = document.getElementById('myUsername').value.trim();
            const ip = document.getElementById('serverIp').value.trim() || 'localhost';

            if (!username) { alert("Digite seu nome!"); return; }

            // Conecta passando o username na URL
            const socket = new SockJS(`http://${ip}:8080/ws?username=${encodeURIComponent(username)}`);
            stompClient = Stomp.over(socket);

            stompClient.connect({}, function () {
                alert('Conectado com sucesso como: ' + username);

                // Inscreve-se na própria fila privada de mensagens
                stompClient.subscribe('/user/queue/private', function (payload) {
                    const msg = JSON.parse(payload.body);
                    const log = document.getElementById('chatLog');
                    log.innerHTML += `<p><strong>${msg.sender} -> ${msg.recipient}:</strong> ${msg.content}</p
>`;
                    log.scrollTop = log.scrollHeight;
                });
            });
        }

        function enviar() {
            const sender = document.getElementById('myUsername').value.trim();
            const recipient = document.getElementById('recipient').value.trim();
            const content = document.getElementById('message').value.trim();

            if (!stompClient) { alert("Conecte-se primeiro!"); return; }

            const payload = { sender: sender, recipient: recipient, content: content };

            // Envia para @MessageMapping("chat") -> /app/chat
            stompClient.send("/app/chat", {}, JSON.stringify(payload));
            document.getElementById('message').value = '';
        }
    </script>
</body>
</html>
```

9. Testando a Aplicação na Prática

Roteiro de Teste no Navegador

Passo a Passo de Execução:

1. Inicie a aplicação Spring Boot na sua IDE executando a classe principal.
2. Abra o navegador e acesse a URL da aplicação: **http://localhost:8080/index.html**
3. Para testar a comunicação entre dois usuários, abra **duas abas anônimas** ou navegadores diferentes:
   - **Aba 1:** Digite User1 no campo 'Seu Nome' e clique em **Conectar**.
   - **Aba 2:** Digite User2 no campo 'Seu Nome' e clique em **Conectar**.
4. Na **Aba 1 (User1)**: Digite User2 no campo 'Destinatário', escreva Olá User2! e clique em **Enviar**.
5. **Resultado:** A mensagem será entregue instantaneamente na **Aba 2 (User2)** e confirmada na tela da **Aba 1**.

Dica para Sala de Aula: Para comunicar com o colega de bancada na mesma rede local (LAN), substitua localhost pelo endereço IP da máquina do professor ou do colega (ex: http://192.168.1.15:8080/index.html).

10. Guia de Referência: Anotações e Componentes do Spring

Anotação / Componente	Tipo	Descrição / Função no Spring
@Configuration	Anotação	Avisa ao Spring que a classe contém definições de Beans de configuração.
@EnableWebSocketMessageBroker	Anotação	Ativa o processamento de mensagens STOMP e a infraestrutura de Message Broker.
WebSocketMessageBrokerConfigurer	Interface	Interface de callback usada para personalizar rotas, prefixos e endpoints.
@Controller	Anotação	Define o componente gerenciador de mensagens WebSocket/STOMP.
@MessageMapping("chat")	Anotação	Mapeia a rota de entrada. Trata pacotes enviados pelos clientes para /app/chat.
@Payload	Anotação	Extrai o corpo da mensagem JSON e desserializa automaticamente no DTO Java.
SimpMessagingTemplate	Componente	Ferramenta injetada para enviar mensagens de forma programática a qualquer momento.
convertAndSendToUser()	Método	Roteia a mensagem exclusivamente para a fila privada do usuário (/user/queue/private).
CustomHandshakeHandler	Classe	Intercepta o handshake HTTP Upgrade para ler ?username= e associar o Principal à sessão.